

しています。1つ目の量子ビットが $|0\rangle$ の状態に CNOT ゲートを作用させると

$$\begin{bmatrix} \alpha_{00} \\ \alpha_{01} \\ 0 \\ 0 \end{bmatrix} \xrightarrow{\text{CNOT}} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} \alpha_{00} \\ \alpha_{01} \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} \alpha_{00} \\ \alpha_{01} \\ 0 \\ 0 \end{bmatrix}$$

と2つ目の量子ビットは変化しない一方で、1つ目の量子ビットが $|1\rangle$ の場合には

$$\begin{bmatrix} 0 \\ 0 \\ \alpha_{10} \\ \alpha_{11} \end{bmatrix} \xrightarrow{\text{CNOT}} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ \alpha_{10} \\ \alpha_{11} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \alpha_{11} \\ \alpha_{10} \end{bmatrix}$$

のように $|10\rangle$ と $|11\rangle$ の確率振幅が入れ替わります。1つ目の量子ビットをコントロール量子ビット、2つ目の量子ビットをターゲット量子ビットと呼びます。ターゲット量子ビットの確率振幅の入れ替えはXゲート操作に相当します。

以上をまとめると、CNOTゲートはコントロールビットが $|1\rangle$ の場合のみターゲット量子ビットを反転させる(Xゲート操作)分岐処理です。古典の場合と異なり、例えば $|0\rangle$ と $|1\rangle$ の重ね合わせ状態をコントロール量子ビットに入力すると

$$\begin{bmatrix} \frac{1}{\sqrt{2}} \\ 0 \\ \frac{1}{\sqrt{2}} \\ 0 \end{bmatrix} \xrightarrow{\text{CNOT}} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} \frac{1}{\sqrt{2}} \\ 0 \\ \frac{1}{\sqrt{2}} \\ 0 \end{bmatrix} = \begin{bmatrix} \frac{1}{\sqrt{2}} \\ 0 \\ 0 \\ \frac{1}{\sqrt{2}} \end{bmatrix} = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

のように、それに応じて出力も重ね合わせ状態で処理されます(コントロール量子ビットの測定による分岐処理ではありません)。ブラケット記法では

$$\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)|0\rangle \xrightarrow{\text{CNOT}} \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

です。CNOTゲートは、古典計算でのXOR(排他的論理和)を可逆にしたもので

$$|ij\rangle \xrightarrow{\text{CNOT}} |i\ (i \text{ XOR } j)\rangle$$

と書けます( $i, j$ は0または1)。量子ゲート操作はすべてユニタリ行列( $U^\dagger U = UU^\dagger = I$ )で、量子ゲート操作はすべて可逆である必要があります。可逆とは出力の状態から入力の状態にいつでも戻れることをいいます。古典のXORゲートは

$$0 \text{ XOR } 0 = 0, \ 0 \text{ XOR } 1 = 1, \ 1 \text{ XOR } 0 = 1, \ 1 \text{ XOR } 1 = 0$$

と作用する2入力1出力の論理ゲートで不可逆です。例えば出力が1の場合に、入力が01と10のどちらだったのかを知る術はありません。

そのため、量子コンピュータでXOR相当のゲートを使うには、出力から入力が必ず逆算できるようにしておく必要があります。ややズルい気もしますが、1つ目の入力値をそのまま出力する2入力2出力のゲートに改造してしまいます。こうすれば、出力されるXOR演算結果と1つの入力値から、もともとの2つの入力値を確実に逆算できるようになります。この改造ゲート $\text{XOR}_R$ は

$$0 \text{ XOR}_R 0 = 00, \ 0 \text{ XOR}_R 1 = 01, \ 1 \text{ XOR}_R 0 = 11, \ 1 \text{ XOR}_R 1 = 10$$

のように作用します。これはまさしくCNOTゲートの振る舞いです。CNOTゲートでは、コントロールビットの情報はそのまま出力され、ターゲットビットにはコントロールビットとのXOR演算の結果が出力されます。古典のXORとの違いは、CNOTゲートでは入力に重ね合わせ状態が許されることです。

量子ビットの状態を入れ替えるSWAPゲートもよく使われます。行列では

$$\text{SWAP} := \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

と定義されます。2量子ビット系の任意の状態 $a_0b_0|00\rangle + a_0b_1|01\rangle + a_1b_0|10\rangle + a_1b_1|11\rangle$ (式(2.1))にSWAPゲート操作を施すと

$$\begin{bmatrix} a_0b_0 \\ a_0b_1 \\ a_1b_0 \\ a_1b_1 \end{bmatrix} \xrightarrow{\text{SWAP}} \begin{bmatrix} a_0b_0 \\ a_1b_0 \\ a_0b_1 \\ a_1b_1 \end{bmatrix} = (b_0|0\rangle_A + b_1|1\rangle_A) \otimes (a_0|0\rangle_B + a_1|1\rangle_B)$$

と確かに量子ビットAとBの確率振幅が入れ替わります。なお、SWAPゲートは

$$\text{SWAP}_{i,j} = \text{CNOT}_{i,j} \text{CNOT}_{j,i} \text{CNOT}_{i,j}$$

とCNOTゲート3つに分解できます(CNOT $_{i,j}$ は量子ビット $i$ をコントロール量子ビット、量子ビット $j$ をターゲット量子ビットとするCNOTゲート)。